

MA 3 - Group Component

December 5, 2025

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import statsmodels.api as sm
import scipy.stats as stats
```

0.1 Problem Statement

Build a linear regression model of the duration of sleep, given the time engaged in physical activity, stress level, and quality of sleep.

0.2 Variables and Parameters

Description	Symbol	Unit
Regression coefficient for intercept	β_0	hour
Time engaged in physical activity	X_1	minute
Regression coefficient for time for physical activity	β_1	$\frac{\text{hour}}{\text{minute}}$
Stress level	X_2	unitless
Regression coefficient for stress level	β_2	hour
Sleep quality	X_3	unitless
Regression coefficient for sleep quality	β_3	hour
Total number of hours slept	Y	hour
Error	ε	hour

0.3 Assumptions and Constraints

- The model follows the linear model $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \varepsilon$.
- The average value of the error is 0: i.e., $\mathbb{E}(\epsilon_i) = 0$ for all i .
- The variance of the error is constant: i.e., $\text{Var}(\epsilon_i) = \sigma^2$ for all i .
- The error $\vec{\epsilon}$ is a random sample from the normal distribution of zero mean and variance σ^2 .
- The error is independent for each ϵ_i for all i .
- Other factors (i.e., plans on the following day) are negligible.
- All participants are independent from one another.
- All data has been collected on the same day.
- Total number of sleep is non-negative.
- The self-reported data are given a clear criteria so there is small variation in values to how they actually feel.

0.4 Building Solutions

We build the linear regression: $Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \varepsilon$.

```
[2]: #import data
sleep = pd.read_csv('Sleep.csv')
sleep = pd.DataFrame(sleep)

[3]: X = sleep[['Quality', 'Physical_Activity', 'Stress']]
Y = sleep['Duration']
X = sm.add_constant(X)
reg = sm.OLS(Y,X).fit()
```

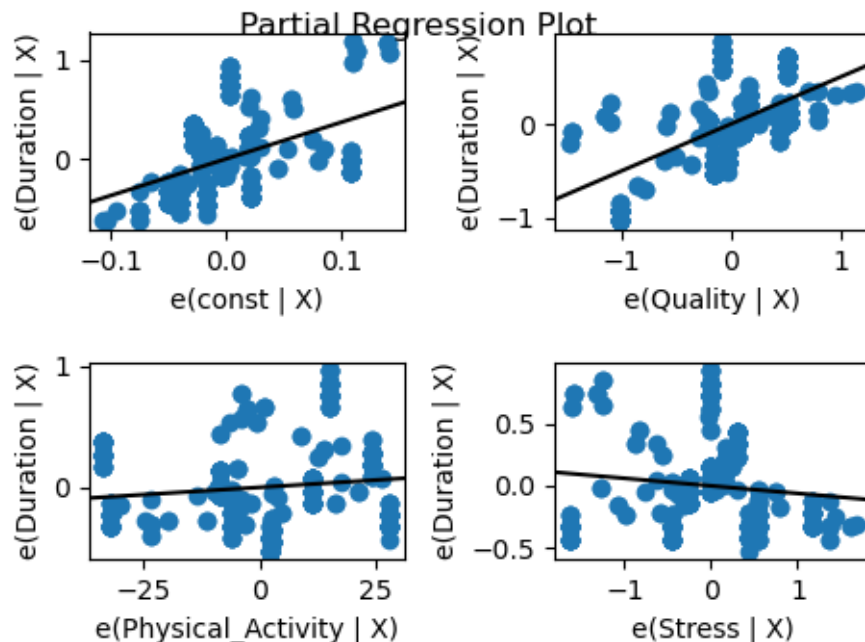
0.5 Analyze and Assess: Transformation

Based on the pre-transformation diagnostics (shown below), we experimented with several transformations suggested in the assignment. Through a process of elimination, we found that applying the transformation to **stress level** and **sleep quality** provided the the best balance between the goodness-of-fit, numerical fit measures, and interpretability. Therefore, we choose to square both variables and name them as following: **Sqr_Stress** and **Sqr_Quality**. This leads to the transformed model:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2^2 + \beta_3 X_3^2 + \varepsilon.$$

We will now explain our choice of transformation by comparing diagnostic plots from the original model and the transformed model.

```
[4]: # Assumption: average value of the error is zero.
sm.graphics.plot_partregress_grid(reg, fig=plt.figure(figsize=(5,3.5)))
plt.show()
```

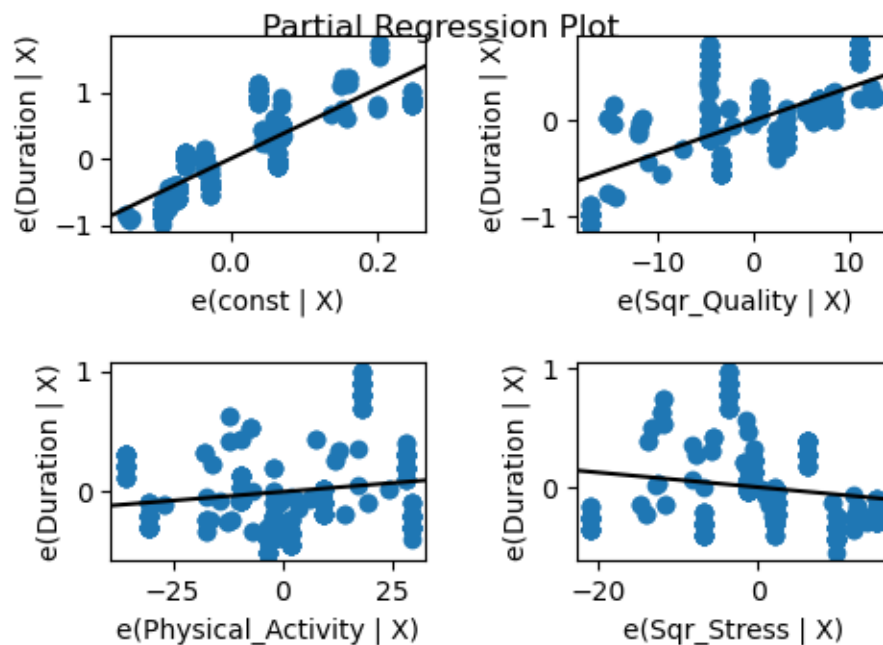


In all four covariate plots, the averages of error are close to zero, but the residual-stress plot and the residual-quality plot seem concerning. The points in the residual-stress plot are not evenly distributed on both sides of the line. There seems to be more points falling below the line than above the line when stress is more than 0. The variance of the distribution of the points doesn't look consistent. Moreover, the residual-quality plot has a lot of points clustered together, and it would be better if we can improve the model to make it scattered out more randomly.

```
[5]: # TRANSFORMATION: sqr(stress), sqr(quality)
sleep["Sqr_Stress"] = sleep["Stress"]**2
sleep["Sqr_Quality"] = sleep["Quality"]**2

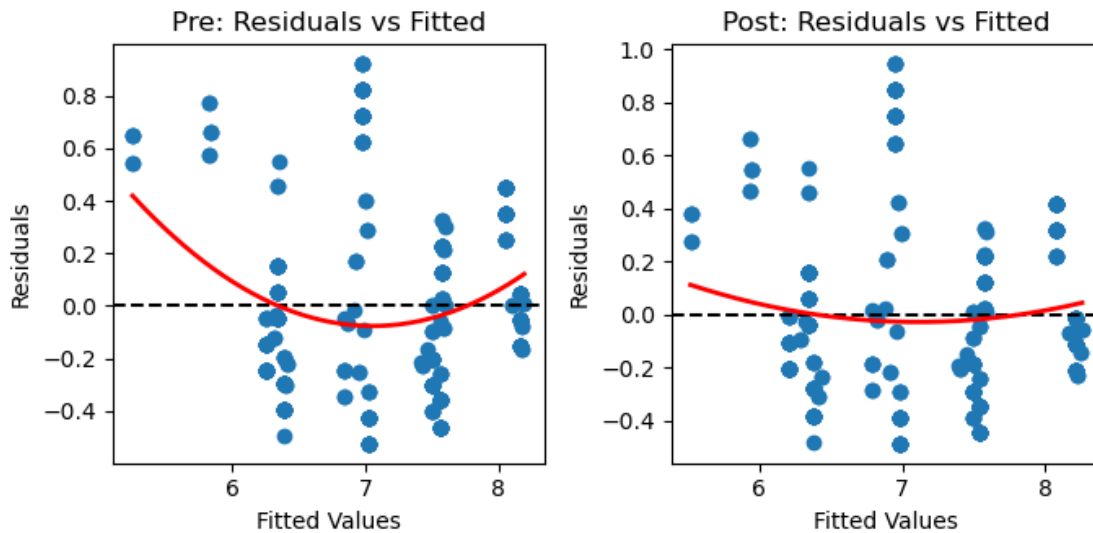
[6]: X2 = sleep[['Sqr_Quality', 'Physical_Activity', 'Sqr_Stress']]
Y2 = sleep[['Duration']]
X2 = sm.add_constant(X2)
reg_2 = sm.OLS(Y2, X2).fit()

[7]: fig1 = sm.graphics.plot_partregress_grid(reg_2, fig=plt.figure(figsize=(5,3.5)))
plt.show()
```



After applying the combination of transformations, there seem to be some improvements on the partial regression plots. A key thing to note here is that, the partial regression plot for the sleep quality shows a more linear pattern with more tightly concentrated points around the linear line. This indicates that the linearity assumption is better satisfied with the transformed model.

```
[8]: models = [(reg, "Pre: Residuals vs Fitted"), (reg_2, "Post: Residuals vs Fitted")]
    fig, ax = plt.subplots(1, 2, figsize=(7,3.5))
    for i, (m, title) in enumerate(models):
        fitted, resid = m.fittedvalues, m.resid
        ax[i].scatter(fitted, resid)
        ax[i].axhline(0, ls='--', color='k')
        ax[i].set_title(title), ax[i].set_xlabel("Fitted Values"), ax[i].
        set_ylabel("Residuals")
        x = np.linspace(fitted.min(), fitted.max(), 200)
        ax[i].plot(x, np.polyval(np.polyfit(fitted, resid, 2), x), color='r',
        linewidth=2)
    plt.tight_layout()
    plt.show()
```



The plot on the left is the Residuals-Fitted plot for the pre-transformed model. In this plot, the points below the line are more condensed than the points above the line, forming a vaguely parabolic trend. Thus it can't be qualified as randomly distributed and violates the assumption that the variance is constant.

In the residuals-fitted plot after transformation (on the right), the centres of the strands of each point are much more flattened out, showing less curvature overall. Although the improvement is not dramatic, this still supports our choice of transformation, as the constant-variance assumption is better satisfied on the transformed model.

After inspecting other diagnostic plots for the transformed model, such as residuals versus each predictor and the QQ-plot, we observe only small changes compared to the non-transformed model. For instance, both QQ-plots have a slight parabolic pattern, and the post-transformation plot only has minimal changes that does not improve the model that much. Because these plots do not cause any new visible violations of linearity, constant variance, or normality, we choose not to include

them here and note that they do not affect our choice of the transformed model.

0.5.1 Observations and Analysis

Important values from the OLS regression results for the original plot is:

```
[9]: out = {"adjR2": reg.rsquared_adj, "cond": reg.condition_number,}
      table = pd.DataFrame({"Coef": reg.params, "t": reg.tvalues, "p": reg.pvalues})
      print(f"Adjusted R\u00b2:", out["adjR2"], '\nCondition Number:', out["cond"])
      print(table)
```

Adjusted R^2 : 0.7833097936534077

Condition Number: 1338.6115150809633

	Coef	t	p
const	3.673707	9.132078	4.459975e-18
Quality	0.498098	12.662634	8.893181e-31
Physical_Activity	0.002414	2.434296	1.539304e-02
Stress	-0.060716	-2.330776	2.030188e-02

Important values from the OLS regression results for the transformed model is:

```
[10]: out = {"adjR2": reg_2.rsquared_adj, "cond": reg_2.condition_number,}
      table = pd.DataFrame({"Coef": reg_2.params, "t": reg_2.tvalues, "p": reg_2.
        ↪pvalues})
      print(f"Adjusted R\u00b2:", out["adjR2"], '\nCondition Number:', out["cond"])
      print(table)
```

Adjusted R^2 : 0.7991617338063545

Condition Number: 890.1292833198715

	Coef	t	p
const	5.280902	28.472838	2.951094e-95
Sqr_Quality	0.034150	14.511770	4.334925e-38
Physical_Activity	0.002941	3.208827	1.449116e-03
Sqr_Stress	-0.006157	-3.029879	2.618472e-03

When we examine the summary table after applying the transformations, the adjusted R^2 increases from 0.783 to 0.799, meaning the new model explains a better fit. This small change is still meaningful because adjusted R^2 is conservative and it provides a strong indication that the transformation improved the model. The transformed model's condition number also decreased substantially (from 1.34×10^3 to 890). Although still high, this represents a meaningful reduction in potential multicollinearity, which is expected because squaring the predictors spreads out their values and weakens linear dependence.

In both models, all predictors remain statistically significant ($p < 0.05$) and the signs are consistent with intuition: higher sleep quality predicts longer duration, higher stress predicts shorter duration, and higher physical activity predicts longer duration. In the transformed model, the coefficients show even stronger significance, with larger t-values, suggesting that the underlying relationships are nonlinear and are better captured by the squared predictors.

Taken together, these results show that the transformed model provides a better fit, reduces multicollinearity, and more accurately represents the nonlinear relationship in the data. The linear

regression is therefore: $\hat{Y} = 5.2809 + 0.0342X_1 + 0.0029X_2^2 - 0.0062X_3^2$.

0.5.2 Model Limitations

By using linear regression we are assuming a linear relationship between our predictors and sleep duration, which may not fully capture the relationship between the variables. If we are being stricter with our linearity assumptions, we can see that some variables may have nonlinear relationships with sleep such as stress. This is shown by the way the stress covariate became more significant after we squared it.

Stress, sleep quality, and activity time are self-reported which introduces potential recall bias and other human measurement errors. Also, quality and stress are likely to be correlated because people who feel more stressed may be more likely to have more trouble sleeping. Since the two variables may be correlated, the model would have difficulty separating their individual effects which may make the coefficients less stable and the p-values less reliable. This does not completely discredit our model but it surely adds a level of uncertainty.

Transformations make our coefficients less intuitive when it comes to interpreting them directly. Squaring our variables increases complexity and could potentially mask true effect. In future iterations of the model, it would be helpful to explore interaction terms, specifically between quality and stress, since realistically they may exist.

Another way to improve our model would be through managing outliers, especially in duration and stress as they may influence the regression coefficients.